

Módulo 07 - Next.js, TypeScript e Integração Frontend/Backend

Objetivo

Estudar como o frontend Next.js foi criado, como passou a conversar com a API Django usando cookies de sessão e CSRF, e como o catálogo real entrou na interface.

Commits Estudados

- `3f2019f` - componentização inicial do site e primeiras telas.
- `b9a9a92` - base HTTP com sessão e CSRF.
- `e4c32d1` - login de cliente integrado com sessão Django.
- `7d2d755` - listagem de brinquedos e kits de festa com busca.
- `3988c20` - melhorias no cabeçalho.
- `391e203` - logo nas telas.
- `683802e` - merge sem diff próprio.
- `f960611` - registro e contador local de carrinho.
- `ebc1814` - padronização de ambiente local em `127.0.0.1`.
- `f25940f` - melhora exibição da logo no menu.
- `ca06f91` - melhora favicon.

Aula 1 - Frontend Inicial Ainda Sem API

O commit `3f2019f` cria o frontend Next.js.

Código para ler:

- `frontend/package.json`
- `frontend/src/app/page.tsx`
- `frontend/src/app/layout.tsx`
- `frontend/src/components/client/Header/index.tsx`
- `frontend/src/components/client/ProductCard/index.tsx`

- `frontend/src/components/ui/*`
- `frontend/src/components/admin/*`

Ponto real do commit:

- O frontend nasce componentizado.
- Existem componentes de UI reutilizáveis.
- Existem telas admin iniciais, mas ainda sem conexão real com API.

Isso é importante porque separa duas fases:

- desenho de interface;
- integração com contrato real do backend.

Aula 2 - Cliente HTTP Com Cookies e CSRF

O commit `b9a9a92` cria a base HTTP.

Código para ler:

- `frontend/src/lib/api.ts`
- `frontend/src/lib/csrf.ts`
- `frontend/src/types/api.ts`

Regras reais:

- `NEXT_PUBLIC_API_BASE_URL` define o backend.
- todas as requisições usam `credentials: "include"`;
- métodos mutáveis buscam e enviam `X-CSRFToken`;
- erro da API é normalizado para `ApiError`;
- respostas `204` são tratadas sem tentar parsear JSON.

Trecho atual:

```
if (MUTABLE_METHODS.has(method)) {
  headers.set("X-CSRFToken", await getCsrfToken());
}

const response = await fetch(buildApiUrl(path), {
  method,
  credentials: "include",
```

```
headers,  
});
```

Ponto crítico:

- Em autenticação por sessão, cookie sozinho não basta.
- CSRF precisa acompanhar requisições mutáveis.

Aula 3 - Login Integrado Com Sessão Django

O commit `e4c32d1` conecta login real.

Código para ler:

- `frontend/src/app/login/page.tsx`
- `frontend/src/contextts/AuthContext.tsx`
- `frontend/src/hooks/useAuth.ts`
- `frontend/src/services/auth.ts`
- `frontend/src/types/auth.ts`
- `frontend/src/app/layout.tsx`

Fluxo real:

1. tela de login envia e-mail e senha;
2. `authService.login` chama `/api/auth/login/`;
3. `apiPost` inclui CSRF e cookies;
4. backend cria sessão;
5. `AuthContext` guarda usuário e cliente;
6. header reage ao estado autenticado.

Ponto de TypeScript:

- Tipos como `LoginPayload`, `AuthMeResponse` e `AuthUser` documentam o contrato com o backend.

Aula 4 - Catálogo Real Com Busca

O commit `7d2d755` conecta a listagem de brinquedos e kits.

Código para ler:

- `frontend/src/app/page.tsx`
- `frontend/src/services/catalogo.ts`
- `frontend/src/types/catalogo.ts`
- `frontend/src/components/client/ProductCard/index.tsx`

Pontos reais:

- Frontend busca brinquedos em `/api/brinquedos/`.
- Frontend busca kits em `/api/kits-festa/`.
- Tipos representam `BrinquedoCatalogo`, `KitFestaCatalogo` e imagens.
- Busca é feita na tela a partir dos dados carregados.

Conexão com backend:

- O frontend exibe somente campos públicos.
- Campo interno como `ativo` não é parte do tipo público.
- `imagem_principal` vem da API criada no commit `62bf4f2`.

Aula 5 - Cabeçalho, Marca e Primeiras Correções Visuais

Os commits `3988c20`, `391e203`, `f25940f` e `ca06f91` ajustam a experiência visual.

Código para ler:

- `frontend/src/components/client/Header/index.tsx`
- `frontend/src/app/login/page.tsx`
- `frontend/public/assets/LogoComEscrita.jpg`
- `frontend/public/assets/SomenteLogo.jpg`
- `frontend/src/app/icon.png`
- `frontend/src/app/apple-icon.png`

Ponto real:

- A marca entra como asset real.
- Header evolui junto com autenticação.
- Favicon e logo são ajustes pequenos, mas importantes para acabamento.

Esses commits não mudam regra de negócio, mas melhoram percepção do produto.

O commit `683802e` é um merge sem diff próprio relevante para estudo técnico. Ele fica registrado na linha do tempo, mas não introduz código novo para analisar neste módulo.

Aula 6 - Registro e Carrinho Inicial no Frontend

O commit `f960611` cria tela de registro e contador local de carrinho.

Código para ler:

- `frontend/src/app/register/page.tsx`
- `frontend/src/services/cart.ts`
- `frontend/src/components/client/Header/index.tsx`
- `frontend/src/components/ui/Input/index.tsx`

Ponto de atenção:

- O contador de carrinho deste commit é local no frontend.
- O backend já tinha carrinho real por sessão desde `7ad2e9c`.
- A integração completa do carrinho frontend/backend ainda é melhoria futura.

Esse é um bom exemplo de decisão incremental: interface começa a representar intenção de produto antes de todo fluxo estar conectado.

Aula 7 - Ambiente Local e Cookies

O commit `ebc1814` padroniza o ambiente local em `127.0.0.1`.

Código para ler:

- `docs/PROJECT_CONTEXT.md`
- `docs/CODEX_RULES.md`
- `frontend/.env.example`
- `frontend/README.md`
- `backend/setup/settings.py`

Regra real:

- frontend: `http://127.0.0.1:3000`;
- backend: `http://127.0.0.1:8000`;
- `NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:8000`;

- não alternar entre `localhost` e `127.0.0.1` no mesmo fluxo.

Motivo técnico:

- sessão, cookie e CSRF dependem de host consistente.

Revisão do Módulo

Você deve conseguir responder:

- Por que `credentials: "include"` é obrigatório?
- Quando o frontend busca CSRF?
- Qual arquivo centraliza chamadas HTTP?
- Como TypeScript ajuda a manter contrato com a API?
- Onde ainda existe diferença entre carrinho visual e carrinho real?

Exercício Prático

Abra `git show b9a9a92` e marque:

- onde a base URL é lida;
- onde cookies entram;
- onde CSRF é adicionado;
- como erro de API é normalizado.

Depois abra `git show 7d2d755` e compare os tipos TypeScript com os serializers públicos do backend.